

Memory-Mapped Interface (MMI) of RISC-V with UART	2
1. Title	2
2. Objective	2
3. Theory (Multi-Level)	2
3.1 Beginner — Simple Intuition.....	2
3.2 Intermediate — Engineering View.....	3
3.3 Advanced — Signal/Architecture Level	3
4. Architecture / Concept.....	4
Block Diagram: MMI Design (No AXI)	4
CPU ↔ UART Communication Flow (Firmware Perspective).....	4
Status Register Polling (uart_getc)	5
5. Implementation.....	6
Files in This Module.....	6
Firmware Key Functions (fw/main.c)	6
6. Lab / Hands-on Section	6
Step 1: Build Firmware Separately (Optional).....	6
Step 2: Run the Full MMI SoC Simulation.....	7
Expected Terminal Output.....	7
Step 3: View Waveforms	7
7. Waveform / Verification Insight.....	8
Key Signals to Monitor.....	8
Observations	8
8. Common Issues & Debug Tips	8
Debugging MMIO: Address Watch	9
9. Summary	9

Memory-Mapped Interface (MMI) of RISC-V with UART

Course: PicoRV32 SoC Subsystem with AXI-Lite & UART Integration

1. Title

Memory-Mapped Interface (MMI) — Connecting PicoRV32 to UART Without AXI

2. Objective

By the end of this module, learners will be able to:

- Explain Memory-Mapped I/O (MMIO) and why it is foundational to embedded systems
- Map UART TX, RX, and STATUS registers to specific memory addresses in software
- Understand how the PicoRV32 `mem_valid/mem_ready` bus translates to peripheral register access
- Run the `picorv32_uart_mem` sub-project and observe CPU ↔ UART communication
- Write simple C firmware that reads/writes UART through fixed memory addresses

3. Theory (Multi-Level)

3.1 Beginner — Simple Intuition

In a typical embedded system, instead of having separate wires for each peripheral command, we use the **memory address space as a control panel**. Each peripheral is assigned a range of addresses:

- Write to address `0x10000000` → Send a byte over UART
- Read from address `0x10000004` → Get the received UART byte
- Read from address `0x10000008` → Check if data is available

This is called **Memory-Mapped I/O (MMIO)**. The CPU doesn't need special hardware for each peripheral — it just uses the same load/store instructions it uses for RAM.

Analogy: Imagine a hotel room with a panel of buttons. Pressing button #1 calls room service, button #2 calls housekeeping, button #3 dims the lights. The CPU presses a “button” (issues a store to an address) and the hardware responds.

3.2 Intermediate — Engineering View

In the `picorv32_uart_mem` design: - PicoRV32 generates `mem_addr + mem_valid + mem_wstrb` for every access - An address decoder routes the request to either ROM, SRAM,

or UART based on the address - The UART peripheral accepts mem_wdata directly without AXI protocol overhead

UART Register Map (from fw/main.c):

```
#define UART_TX      (*(volatile char*)0x10000000) // Write → transmit byte
#define UART_RX      (*(volatile char*)0x10000004) // Read  → received byte
#define UART_STATUS  (*(volatile int*)0x10000008)  // Read  → [1]=RX ready
```

volatile keyword: Tells the C compiler not to optimize away hardware register accesses. Without it, the compiler might cache the value in a CPU register and never actually perform the memory-mapped access.

3.3 Advanced — Signal/Architecture Level

Address Range Decoding (direct mem-mapped, no AXI):

CPU addr [31:0] → Combinational decode:

```
[31:16] == 16'h0000 AND [15] == 0 → ROM   (0x0000_0000 - 0x0000_7FFF)
[31:16] == 16'h0001                → SRAM  (0x0001_0000 - 0x0001_FFFF)
[31:28] == 4'h1                    → UART  (0x1000_0000 - 0x1000_000F)
```

UART access via direct mem bus:

```
// mem_uart: Write side
if (mem_valid && (!mem_wstrb) && uart_sel) begin
    uart_tx_data <= mem_wdata[7:0]; // grab byte from CPU write
    uart_tx_valid <= 1'b1;          // trigger UART TX
end

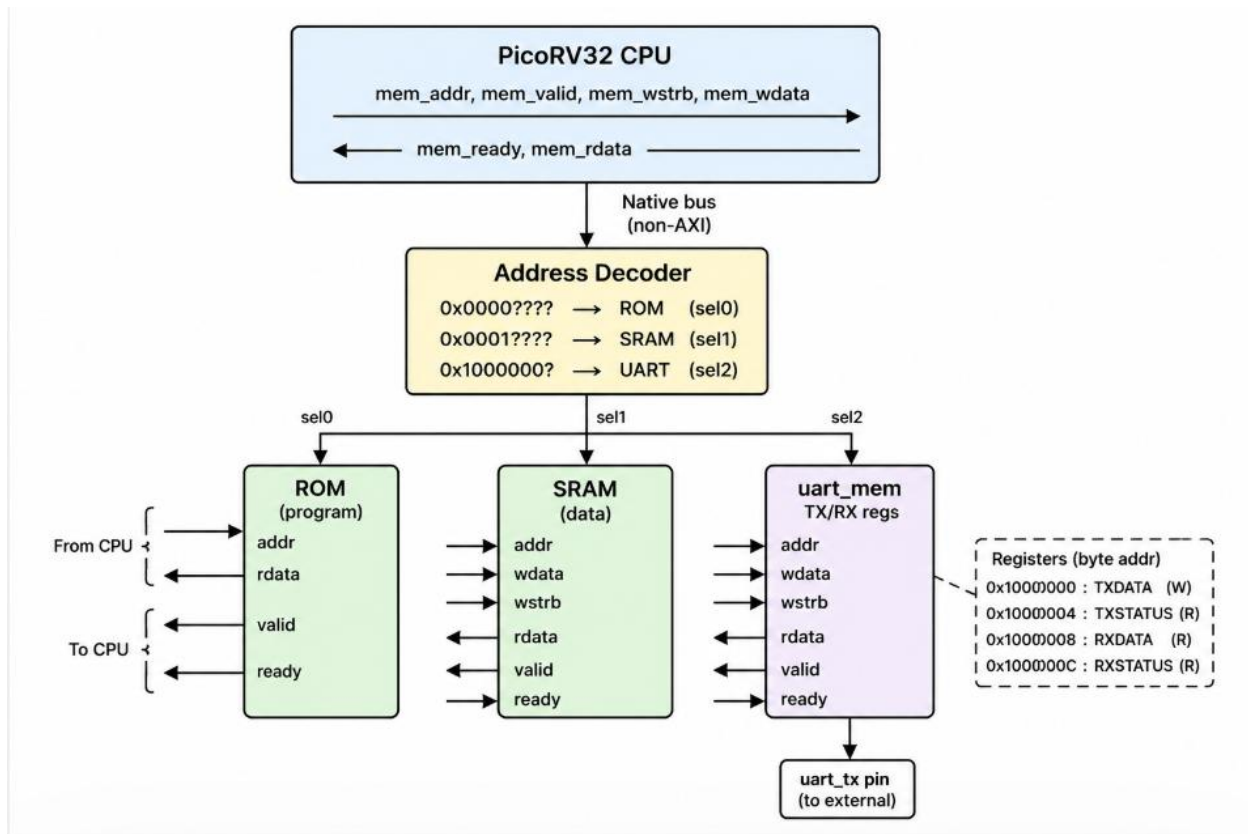
// mem_uart: Read side
if (mem_valid && (!mem_wstrb) && uart_sel) begin
    if (mem_addr[3:0] == 4'h8)
        mem_rdata <= {30'b0, rx_valid, 1'b0}; // status register
    else
        mem_rdata <= {23'b0, rx_valid, rx_data}; // RX data register
end
```

System memory map (full picture):

Address Range	Peripheral	Interface
0x0000_0000 - 0x0000_7FFF	ROM (program)	Direct native bus
0x0001_0000 - 0x0001_FFFF	SRAM (data)	Direct native bus
0x1000_0000 - 0x1000_000F	UART	Direct native bus

4. Architecture / Concept

Block Diagram: MMI Design (No AXI)



CPU ↔ UART Communication Flow (Firmware Perspective)

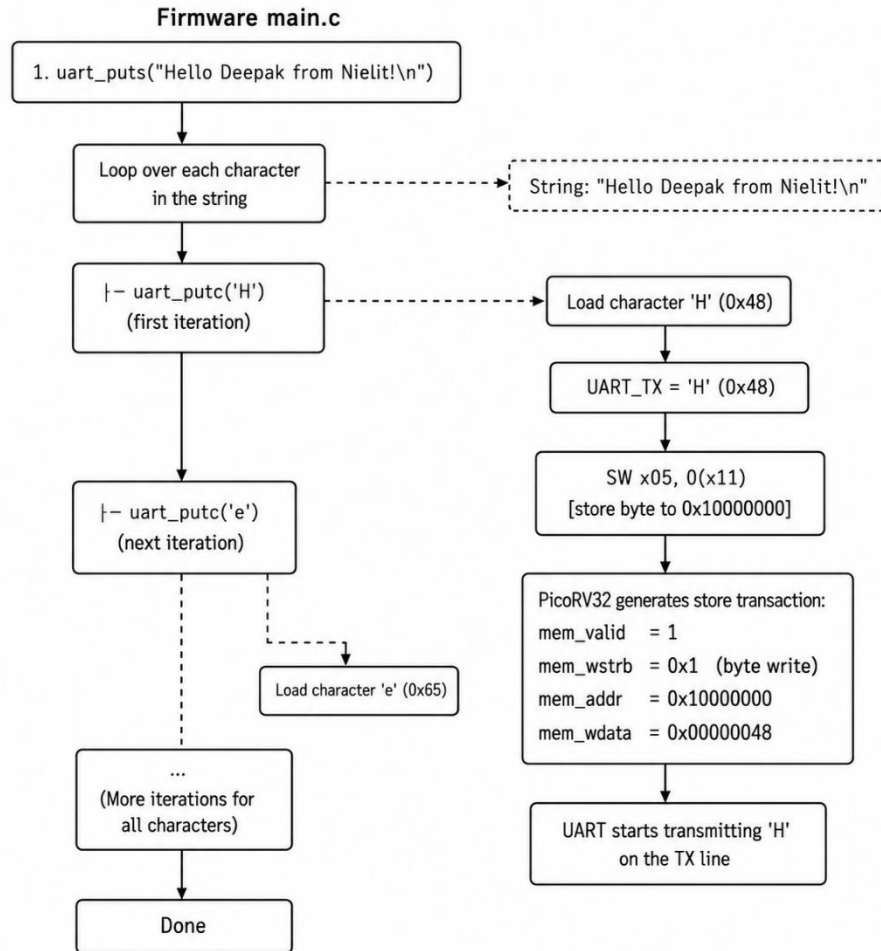
firmware main.c:

```
1. uart_puts("Hello Deepak from Nielit!\n")
```

```

├─ uart_putc('H')
│   └─ UART_TX = 'H'
│       └─ SW x05, 0(x11) [store byte to 0x10000000]
│           └─ PicoRV32: mem_valid=1, mem_wstrb=0x1,
mem_addr=0x10000000
│               mem_wdata=0x48
│                   → UART starts transmitting 'H'
├─ uart_putc('e') (next iteration)
└─ ...

```



Status Register Polling (`uart_getc`)

```

while (!(UART_STATUS & 0x2)); // Poll bit 1 = RX ready
return UART_RX;               // Read received byte

```

```

LW a0, 8(uart_base) ; load status register
ANDI a0, a0, 2 ; check bit 1 (RX ready)
BEQ a0, x0, loop ; retry if not ready
LW a1, 4(uart_base) ; load received byte

```

5. Implementation

Files in This Module

File	Role
<code>picorv32_uart_mem/rtl/top.v</code>	MMI SoC top — CPU + ROM + SRAM + UART (direct bus)
<code>picorv32_uart_mem/rtl/uart_mem.v</code>	Direct-bus UART peripheral (no AXI)
<code>picorv32_uart_mem/rtl/mem_rom.v</code>	ROM with native memory interface
<code>picorv32_uart_mem/rtl/mem_sram.v</code>	SRAM with native memory interface

File	Role
fw/main.c	Firmware: prints greeting 10× then echo loop
fw/crt0.S	C runtime startup: stack init + call main
fw/link.ld	Linker script: ROM from 0x0, STACK at 0x1_00FC
picorv32_uart_mem/tb/tb_mem_soc.v	Testbench: UART monitor + PING test

Firmware Key Functions (fw/main.c)

```
#define UART_TX      (*(volatile char*)0x10000000)
#define UART_RX      (*(volatile char*)0x10000004)
#define UART_STATUS  (*(volatile int*)0x10000008)

static void uart_putc(char c) { UART_TX = c; }

static void uart_puts(const char *s) {
    while (*s) uart_putc(*s++);
}

static char uart_getc(void) {
    while (!(UART_STATUS & 0x2)); // Wait for RX ready
    return UART_RX;
}

int main(void) {
    for (int i = 0; i < 10; i++)
        uart_puts("Hello Deepak from Nielit!\n");
    while (1) uart_putc(uart_getc()); // Echo Loop
}
```

6. Lab / Hands-on Section

Step 1: Build Firmware Separately (Optional)

```
cd ~/Desktop/OpenLaneUser/designs/picorv32_soc
make
# Produces: firmware.elf, firmware.bin, rom.hex
```

The firmware build chain:

```
main.c + crt0.S → (riscv32-unknown-elf-gcc) → firmware.elf
firmware.elf → (objcopy) → firmware.bin
firmware.bin → (bin2hex.py) → rom.hex (for $readmemh in RTL)
```

Step 2: Run the Full MMI SoC Simulation

```
cd /home/deep/nielit/projects/tested_projects/picorv32_soc
make soc_pico_uart_mem
```

This command: 1. Builds firmware in picorv32_uart_mem/fw/ 2. Compiles the MMI SoC RTL + testbench with Verilator 3. Runs the simulation → produces tb_mem_soc.vcd/.fst 4. Opens GTKWave automatically

Expected Terminal Output

```
-----
UART SoC Simulation Started
-----

[TB] UART Monitor Started
-----
[TB] RX byte: H
[TB] RX byte: e
[TB] RX byte: l
[TB] RX byte: l
[TB] RX byte: o
...
[TB] Greeting message count = 10

=====
10 GREETING MESSAGES RECEIVED
UART OUTPUT VERIFIED
=====

-----
[TB] FIFO flushed - sending command : PING
-----
[TB] RX byte: P
[TB] RX byte: I
[TB] RX byte: N
[TB] RX byte: G

=====
TEST PASSED
Echo received : PING
=====
```

Step 3: View Waveforms

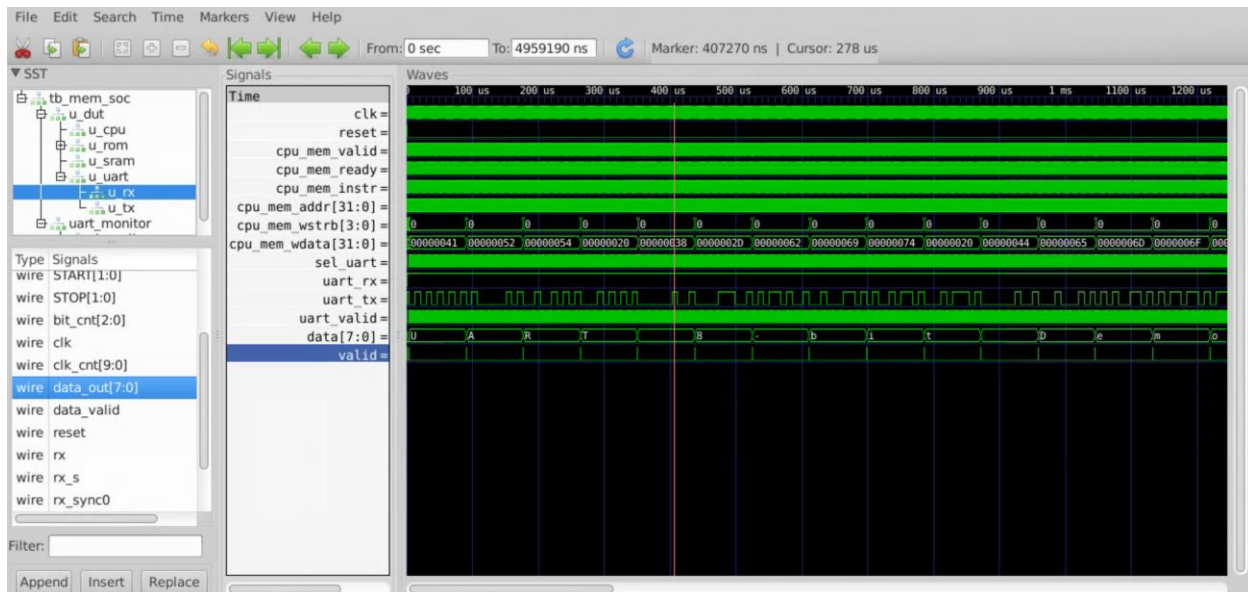
make wave *# Opens tb_top.fst (or tb_mem_soc.fst for this target)*

7. Waveform / Verification Insight

Key Signals to Monitor

clk
reset
cpu_mem_valid
cpu_mem_instr
cpu_mem_ready
cpu_mem_addr[31:0]

cpu_mem_wstrb[3:0]
cpu_mem_wdata[31:0]
uart_sel ← goes high when addr = 0x10000000
uart_tx ← serial bitstream output
uart_mem.tx_valid ← TX trigger from CPU
uart_mem.rx_data_valid ← RX byte received



Observations

- Instruction fetches:** cpu_mem_instr=1, addr incrementing through ROM space (0x0000_0004, 0x0000_0008, ...)
- UART TX write:** cpu_mem_valid=1, cpu_mem_wstrb=4'b0001, cpu_mem_addr=0x1000_0000, cpu_mem_wdata[7:0] = 0x48 ('H')
- UART byte serialized:** After the write, uart_tx transitions from 1→0 (start bit) then through 8 data bits over 4340 clock cycles
- Status poll:** CPU repeatedly reads 0x1000_0008 until bit 1 is set (PING phase)
- Echo path:** CPU reads 0x1000_0004, then writes it back to 0x1000_0000

8. Common Issues & Debug Tips

Issue	Likely Cause	Fix
soc_pico_uart_mem fails to build	Firmware not compiled	Run <code>cd fw && make first</code>
0 greeting messages received	ROM not loaded	Check <code>+romhex=</code> plusarg path in Makefile
UART TX flat (never transitions)	CPU never reaches <code>uart_putc</code> call	Check stack and PC trace — bad linker script?
Status poll hangs forever	uart_status bit mapping wrong	Verify UART_STATUS bit 1 = RX ready in RTL

Issue	Likely Cause	Fix
Echo sends wrong character	UART_RX reads stale data	Ensure rx_buf_valid clears after read
Simulation times out	CPU in infinite wait loop	Add \$display monitoring of cpu_mem_addr to track CPU state

Debugging MMIO: Address Watch

In the testbench, add this always block for immediate feedback:

```
always @(posedge clk) begin
    if (mem_valid && mem_addr == 32'h10000000 && |mem_wstrb)
        $display("[DBG] UART TX write: 0x%02h ('%c')", mem_wdata[7:0],
mem_wdata[7:0]);
end
```

9. Summary

Concept	Detail
MMI	Peripherals mapped to CPU address space; accessed via load/store
UART registers	TX: 0x1000_0000
volatile in C	Prevents compiler from caching hardware register reads
Address decode	Combinational logic maps address ranges to peripheral select signals
Blocking RX	uart_getc() polls STATUS bit 1 until RX data is available
Lab result	CPU prints 10× “Hello Deepak from Nielit!” then echoes “PING” → TEST PASSED
vs Sub-module 5+	This Sub-module uses a direct native bus ; Sub-module 5+ adds full AXI-Lite protocol

This Sub-module bridges the gap between raw RISC-V ISA knowledge (Sub-module 1) and the full AXI-based SoC (Sub-modules 5–8). Understanding MMIO is essential for all embedded firmware development.

Previous: [Sub-module 3 — UART](#)

Next: [Sub-module 5 — RISC-V AXI Integration](#)